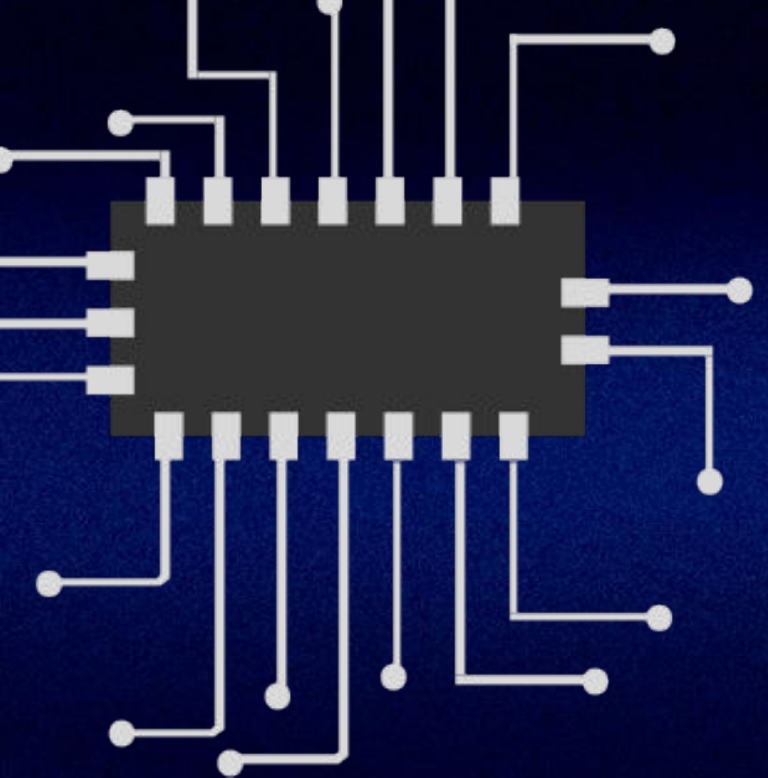




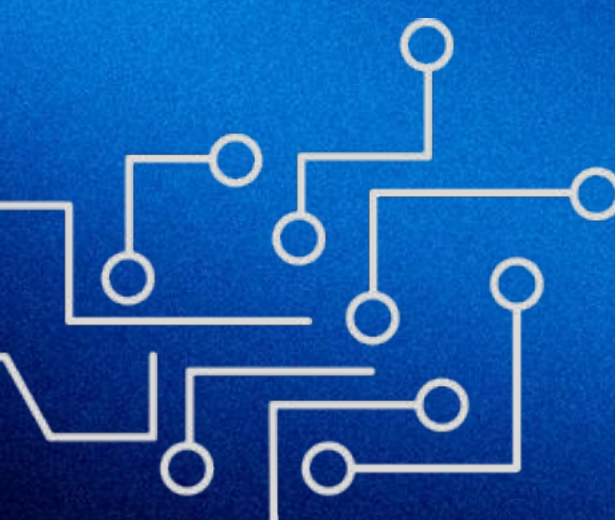
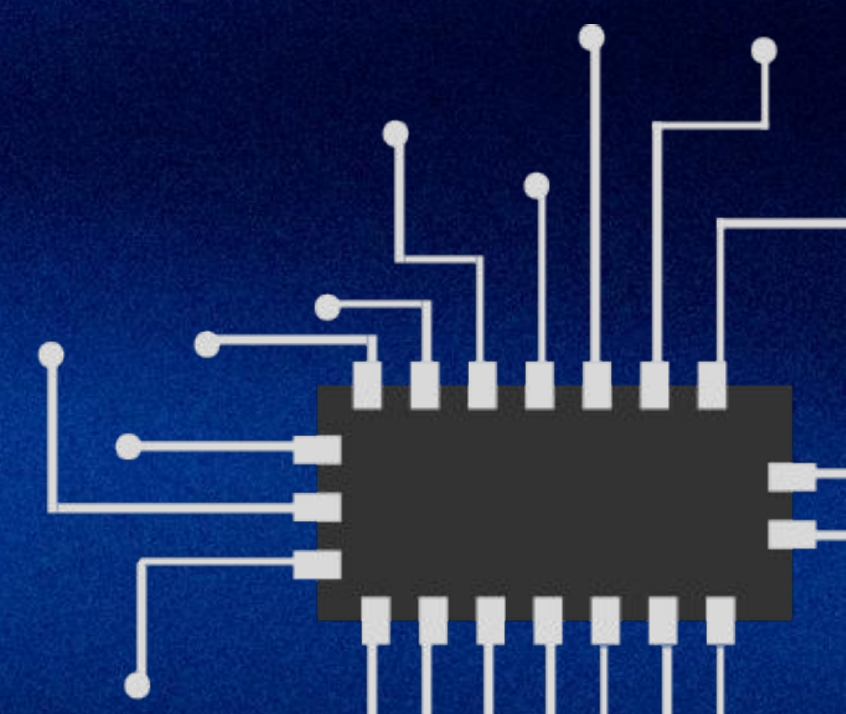
ASIC FOR MICROCONTROLLER MEMORY TESTING

Presented by Team 9:

Aardra S V

G Mary Priya

Luba Maiza

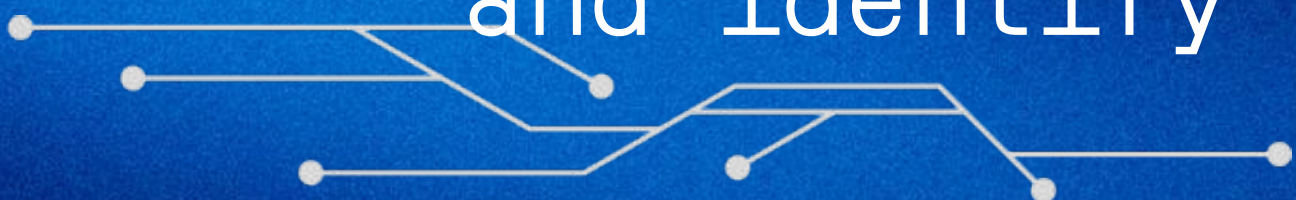




PROJECT OVERVIEW



- This project focuses on implementing an ASIC aimed at monitoring the data integrity of microcontrollers and memory systems in real-time.
- We have utilized the principles of parity generation and error decoding to detect and identify bit-level faults.

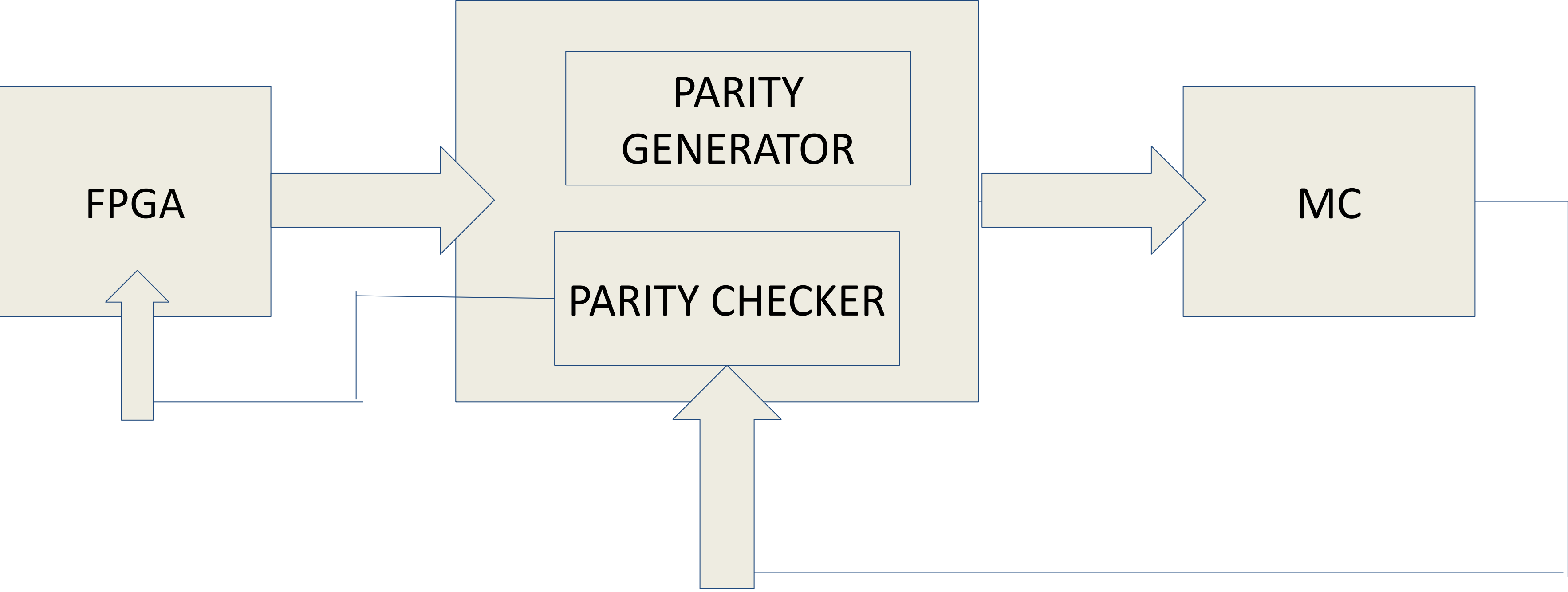




METHODOLOGY

- We have implemented this ASIC using the (7,4) hamming code methodology.
- The ASIC contains 2 main blocks: Parity generator and Parity checker.
- The Parity generator takes 4-bit data input and produces 7-bit hamming code as output.
- The Parity checker takes 7-bit hamming code input and checks for error and gives 3-bit error output.
- The ASIC is designed to detect single bit error (single bit flip) in the hamming code.







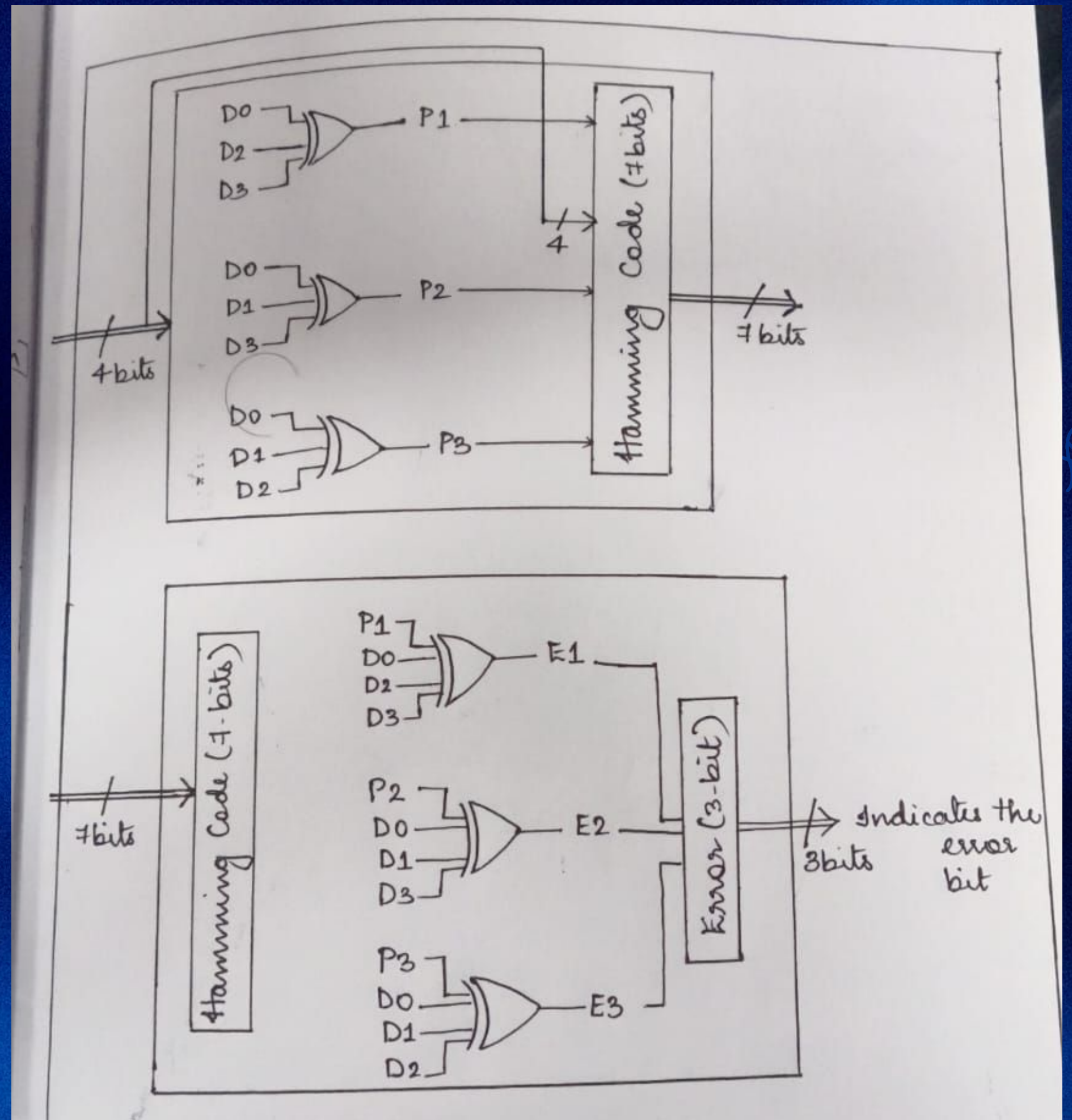
PROCESS

1. Simulation (xcelium)
2. Synthesis (genus)
3. Floorplanning (innovus)
4. Powerplanning (innovus)
5. Placement (innovus)
6. Clock Tree Synthesis (innovus)
7. Routing (innovus)
8. Signoff (innovus)



BLOCK

DIAGRAM





1.VERILOG

- We created 3 modules: Top, parity_gen and parity_checker
 - The top module selects which operation is to be done; parity generation or parity checking. Both the parity_gen and parity_checker modules are instantiated in the top module
 - The parity_gen module generated 7 bit hamming code and sends it out as output. A 'ready' flag is raised once the hamming code is produced.
 - The parity_checker module checks the received hamming code for any error and gives the error as output.
 - In the test bench we produced hamming code for binary 0 to 15 and also checked the parity_checker by giving 3 different hamming codes with and without error.
-

Top module

```
localparam
gen = 1'b0,
check = 1'b1;

reg state_reg, state_next;
always @(posedge clk or posedge rst) begin
if (rst)
    state_reg <= gen; // default to generator mode
else
    state_reg <= state_next;
end
always @(*) begin
    state_next = state_reg; // Default: stay in current state
    case (state_reg)
        gen: begin
            if (ready)
                state_next = check;
            else
                state_next = gen;
            end
        check: state_next = gen;
    endcase
end
```

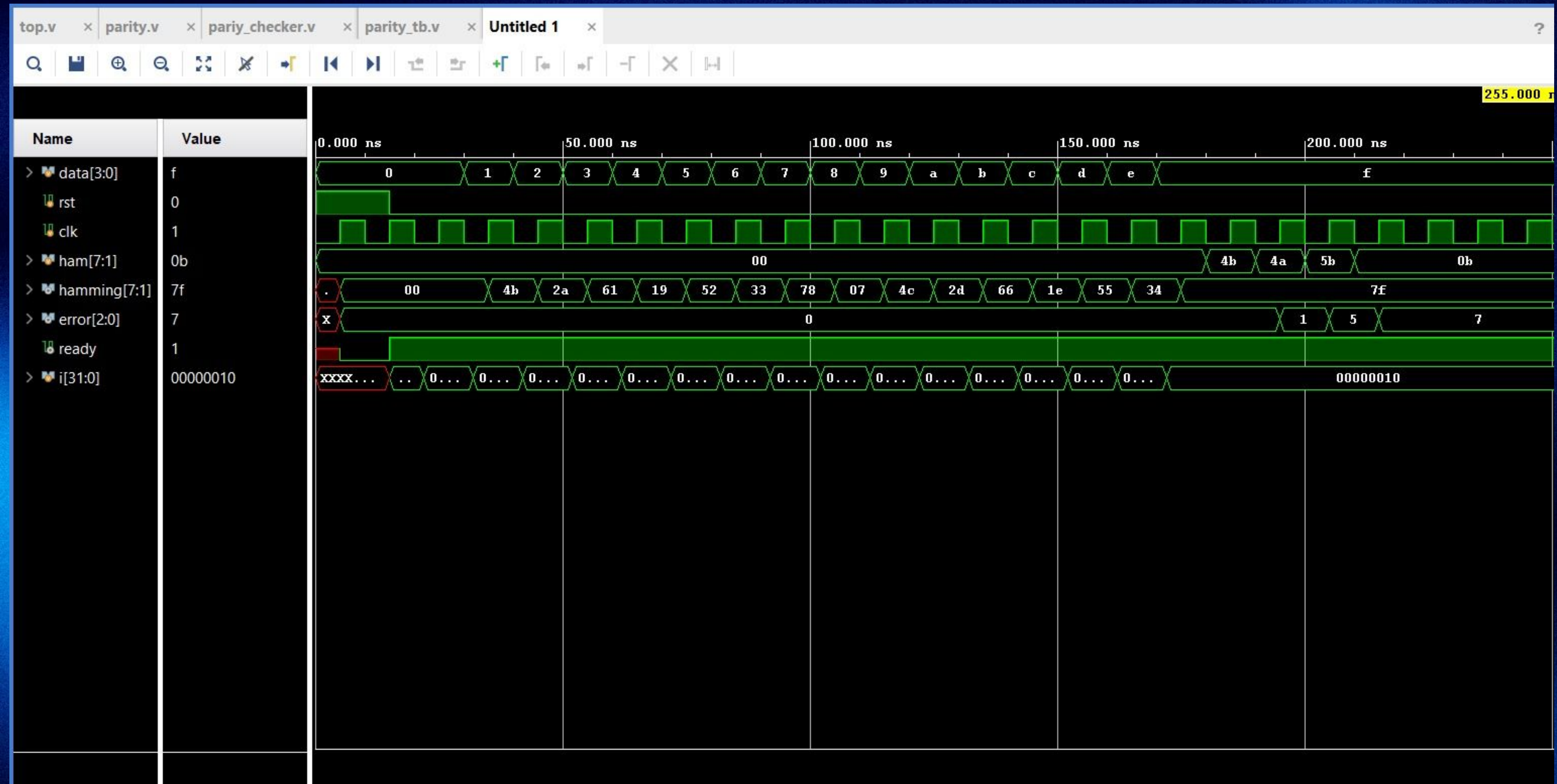
Parity generator

```
always @(posedge clk) begin
    if(rst)begin
        hamming <= 7'b0000000;
        ready = 1'b0;
    end
    else begin
        hamming[1] <= data[0]^data[2]^data[3];
        hamming[2] <= data[0]^data[1]^data[3];
        hamming[3] <= data[3];
        hamming[4] <= data[0]^data[1]^data[2];
        hamming[5] <= data[2];
        hamming[6] <= data[1];
        hamming[7] <= data[0];
        ready <= 1'b1;
    end
end
```

Parity checker

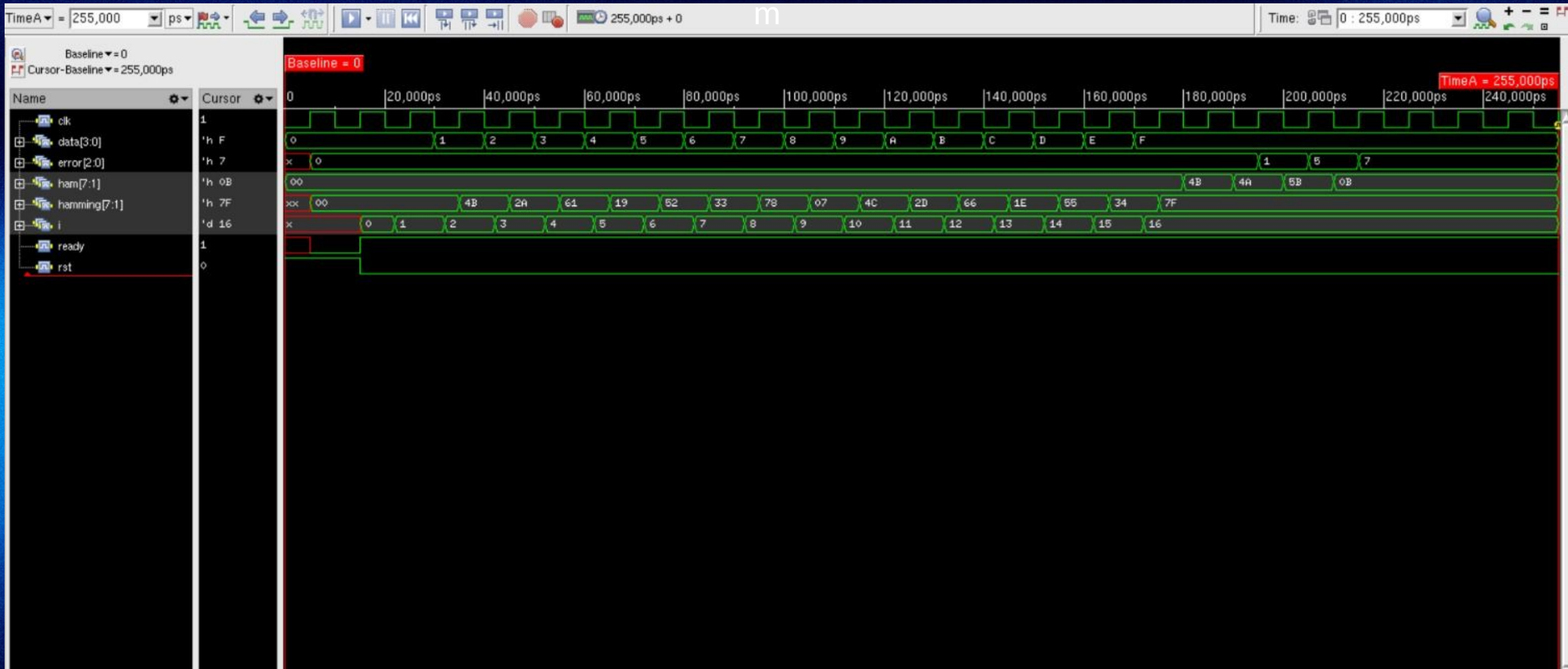
```
always @(posedge clock)begin
    if(reset)begin
        error <= 3'b000;
    end
    else begin
        E[1] = ham[1]^ham[3]^ham[5]^ham[7];
        E[2] = ham[2]^ham[3]^ham[6]^ham[7];
        E[3] = ham[4]^ham[5]^ham[6]^ham[7];
        error = E;
    end
end
```


Test bench



2.SIMULATION

Xceliu



3.SYNTHESIS

- We added the clock pad, input pads and output pads in a new file called parity_main.v
- The run.tcl file was edited.
- Cadence tool - genus was invoked and run.tcl was sourced.
- design_area, design_timing, design_power, design_gates reports were generated.

```
//clock
pc3d01 pc3d01_1 (.PAD(clk_pad), .CIN(parity_clk));
pc3c01 pc3c01_1 (.CCLK(parity_clk), .CP(clk));

//reset instantiation
pc3d01 pc3d01_2 (.PAD(rst_pad), .CIN(rst));

//ham instantiation
pc3d01 pc3d01_3 (.PAD(ham_pad[1]), .CIN(ham[1]));
pc3d01 pc3d01_4 (.PAD(ham_pad[2]), .CIN(ham[2]));
pc3d01 pc3d01_5 (.PAD(ham_pad[3]), .CIN(ham[3]));
pc3d01 pc3d01_6 (.PAD(ham_pad[4]), .CIN(ham[4]));
pc3d01 pc3d01_7 (.PAD(ham_pad[5]), .CIN(ham[5]));
pc3d01 pc3d01_8 (.PAD(ham_pad[6]), .CIN(ham[6]));
pc3d01 pc3d01_9 (.PAD(ham_pad[7]), .CIN(ham[7]));

//data_in instantiation
pc3d01 pc3d01_10 (.PAD(data_pad[0]), .CIN(data[0]));
pc3d01 pc3d01_11 (.PAD(data_pad[1]), .CIN(data[1]));
pc3d01 pc3d01_12 (.PAD(data_pad[2]), .CIN(data[2]));
pc3d01 pc3d01_13 (.PAD(data_pad[3]), .CIN(data[3]));

//hamming instantiations
pc3o05 pc3o05_1 (.I(hamming[1]), .PAD(hamming_pad[1]));
pc3o05 pc3o05_2 (.I(hamming[2]), .PAD(hamming_pad[2]));
pc3o05 pc3o05_3 (.I(hamming[3]), .PAD(hamming_pad[3]));
pc3o05 pc3o05_4 (.I(hamming[4]), .PAD(hamming_pad[4]));
pc3o05 pc3o05_5 (.I(hamming[5]), .PAD(hamming_pad[5]));
pc3o05 pc3o05_6 (.I(hamming[6]), .PAD(hamming_pad[6]));
pc3o05 pc3o05_7 (.I(hamming[7]), .PAD(hamming_pad[7]));

//error instantiation
pc3o05 pc3o05_8 (.I(error[0]), .PAD(error_pad[0]));
pc3o05 pc3o05_9 (.I(error[1]), .PAD(error_pad[1]));
pc3o05 pc3o05_10 (.I(error[2]), .PAD(error_pad[2]));

//ready
pc3o05 pc3o05_11 (.I(ready), .PAD(ready_pad));
```


Information from the reports:

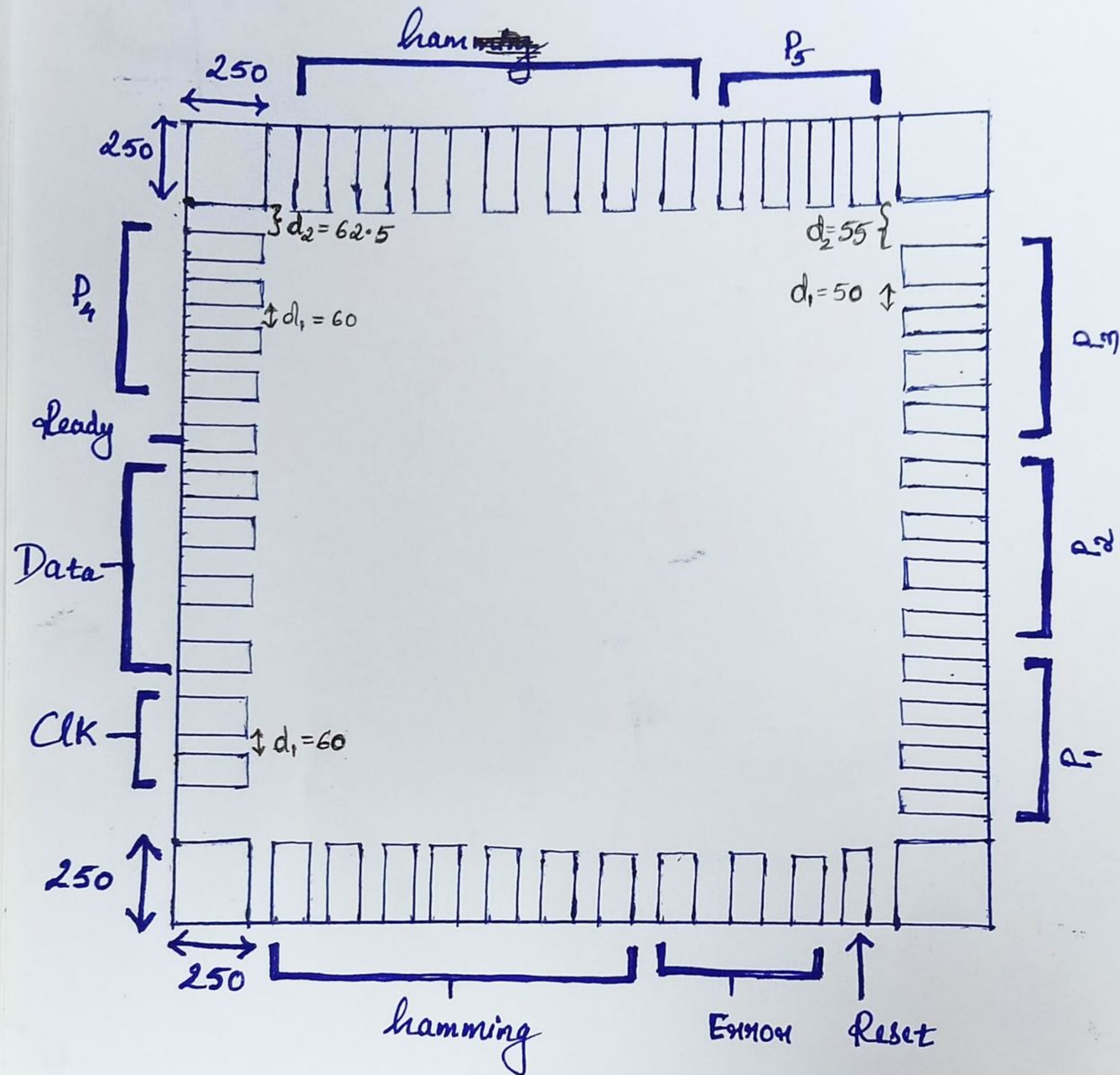
- Library used: scl 180nm
 - Total area = 33531 square micro-meter
 - Total of 60 physical cells
 - Slack = 2015ps positive
 - Data arrival time - 7.985 ns
 - Data required time - 10.000ns
 - Timing status met.
 - Operating condition - Worst case
 - Total power consumption - 2.497mW
-



4.FLOORPLAN

- We have a total of 49 pads (25 i/o pads, 20 power pads, 4 corner pads).
- We decided to give 11 pads each on 3 sides of the ASIC and 12 pads on the fourth side.
- The distance between each pads (on the sides with 11 pads) was calculated and was equal to 60micro-meters and the distance between the corner pad and the i/o pad was 62.5 micro-meters.
- The distance between the pads (on the side with 12 pads) was 50 micro-meters and the distance between the corner pad and i/o pad was 55 micro-meters.





For 11 pins

$$65 \times 11 + 10d_1 + 2d_2 + 500 = 1940 \quad \because d_1 = d_2 = d$$

$$715 + 12d + 500 = 1940$$

$$1215 + 12d = 1940$$

$$12d = 725$$

$$d = 60.416$$

$$d \approx 60$$

Substituting d in d_1

$$715 + 10 \times 60 + 2d_2 + 500 = 1940 \quad \because d_1 = 60$$

$$715 + 600 + 2d_2 + 500 = 1940$$

$$1815 + 2d_2 = 1940$$

$$2d_2 = 125$$

$$d_2 = 62.5$$

$$\therefore; \boxed{d_1 = 60} \text{ \& \& } \boxed{d_2 = 62.5}$$

For 12 pins

$$65 \times 12 + 11d_1 + 2d_2 + 500 = 1940$$

$$11d_1 + 2d_2 = 660$$

$$\text{Assume } d_1 = d_2 = d$$

$$13d = 660$$

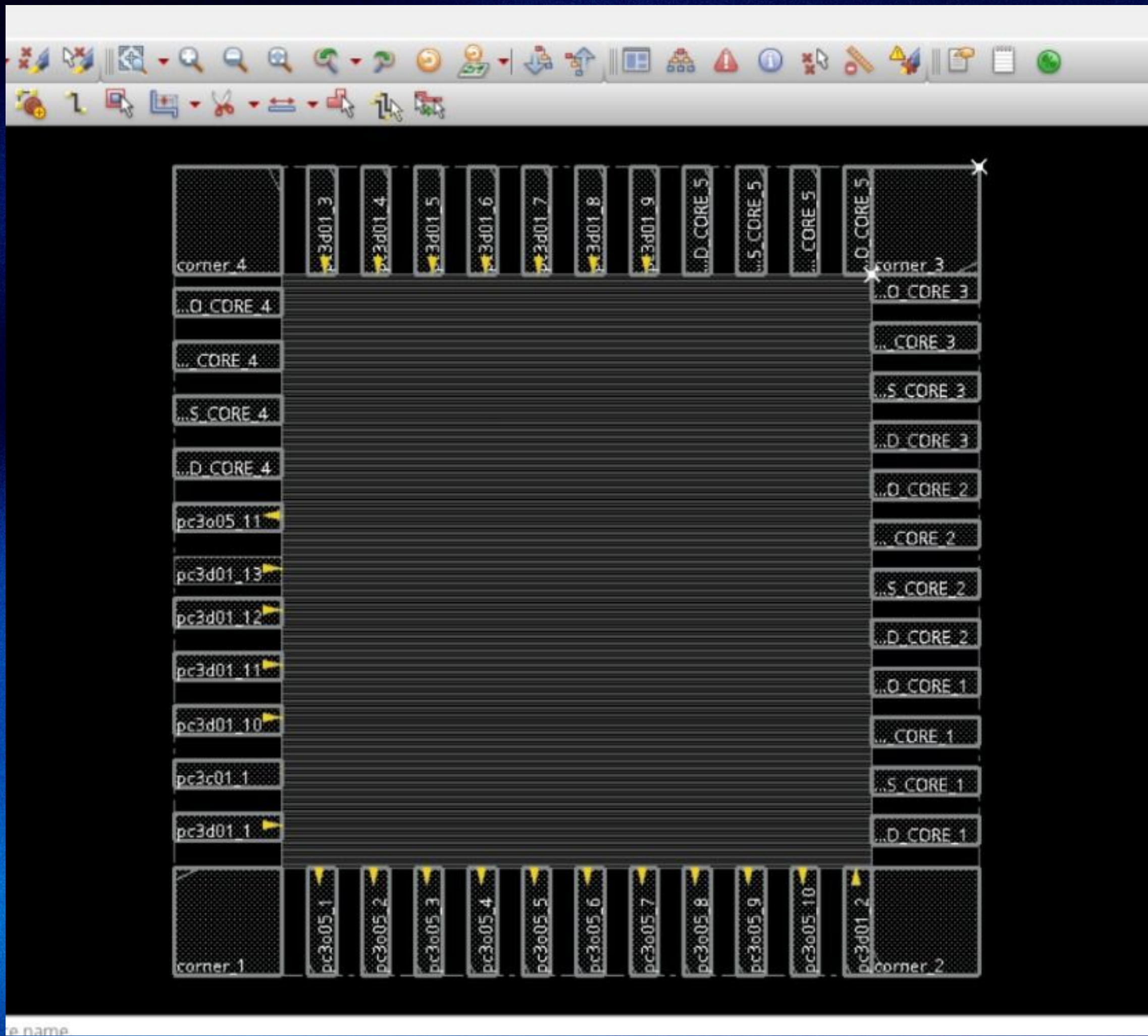
$$d = 50.76$$

$$\text{Let } d_1 = 50$$

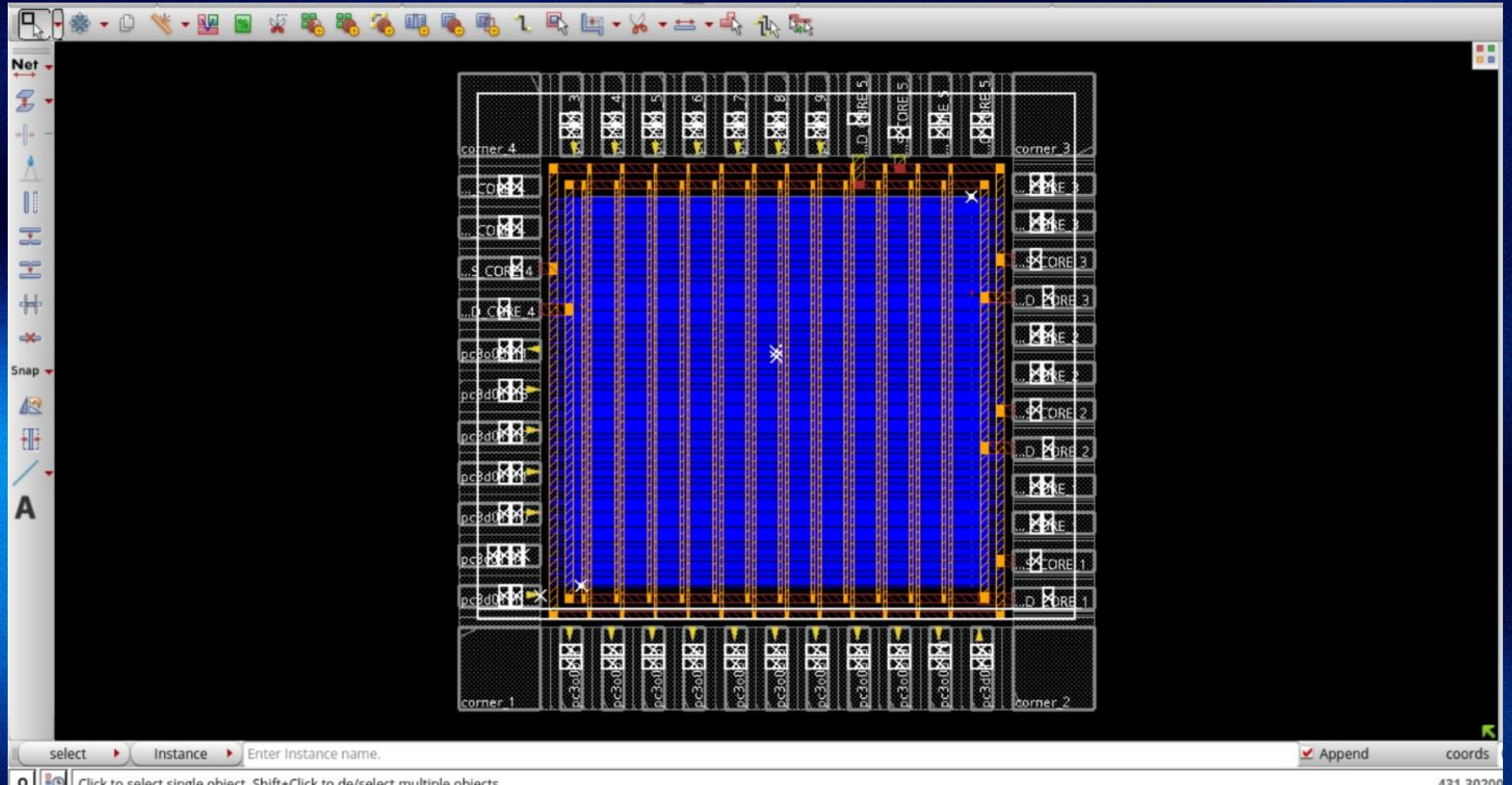
$$11 \times 50 + 2d_2 = 660$$

$$d_2 = 55$$

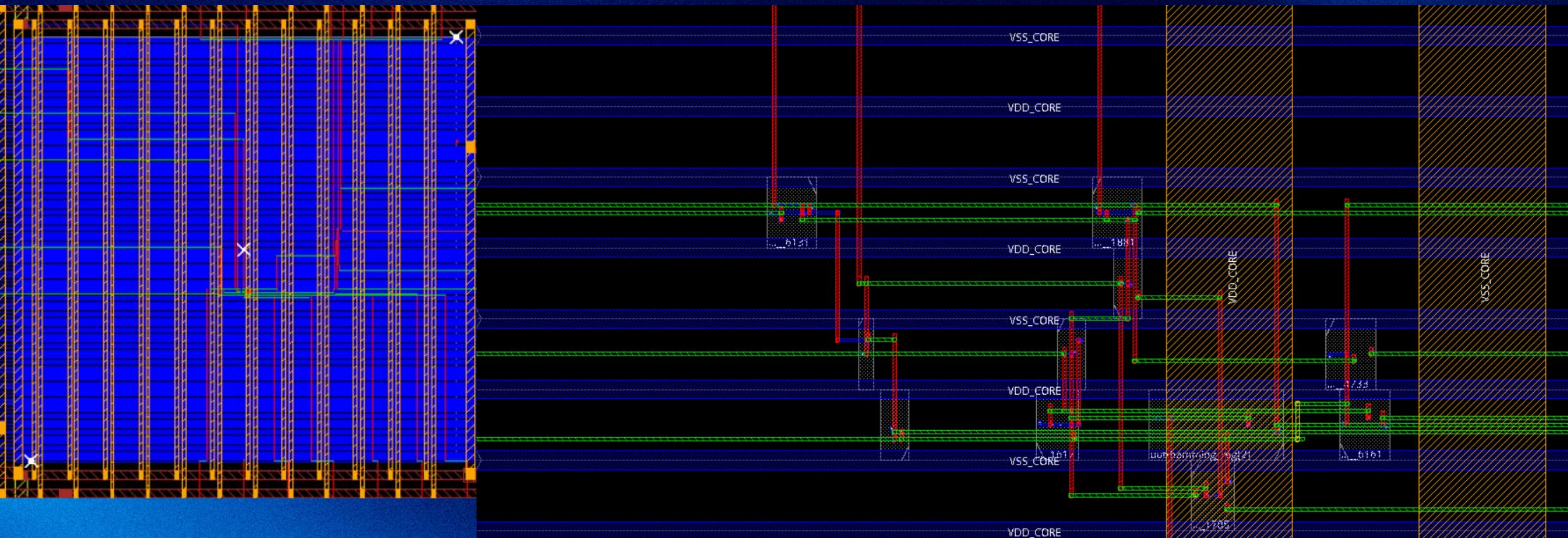
$$\therefore; \cancel{d_1 = 50.76} \quad \boxed{d_1 = 50} \text{ \& \& } \boxed{d_2 = 55}$$



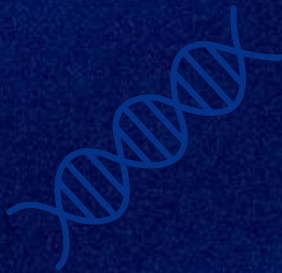
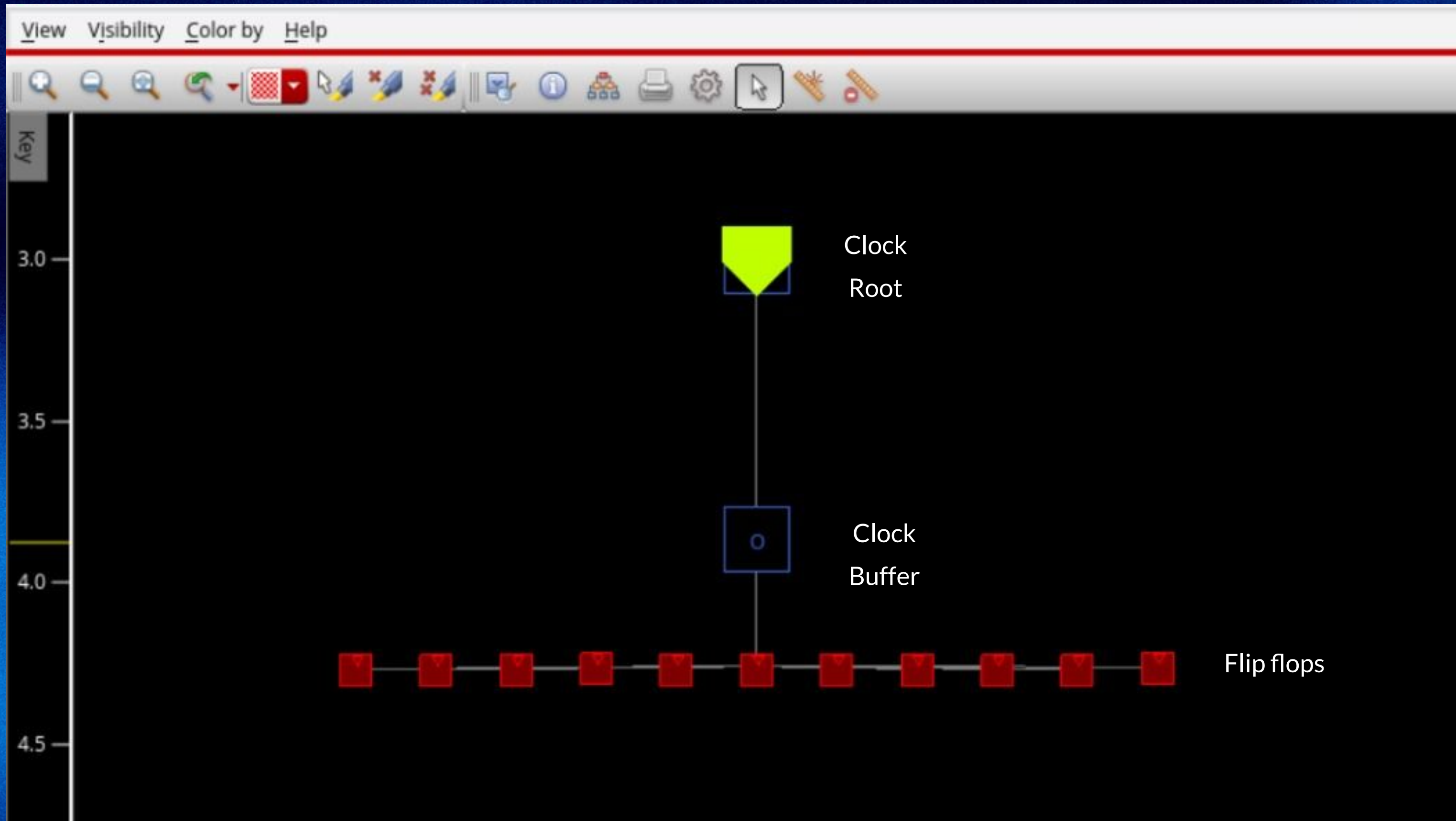
5. POWERPLAN



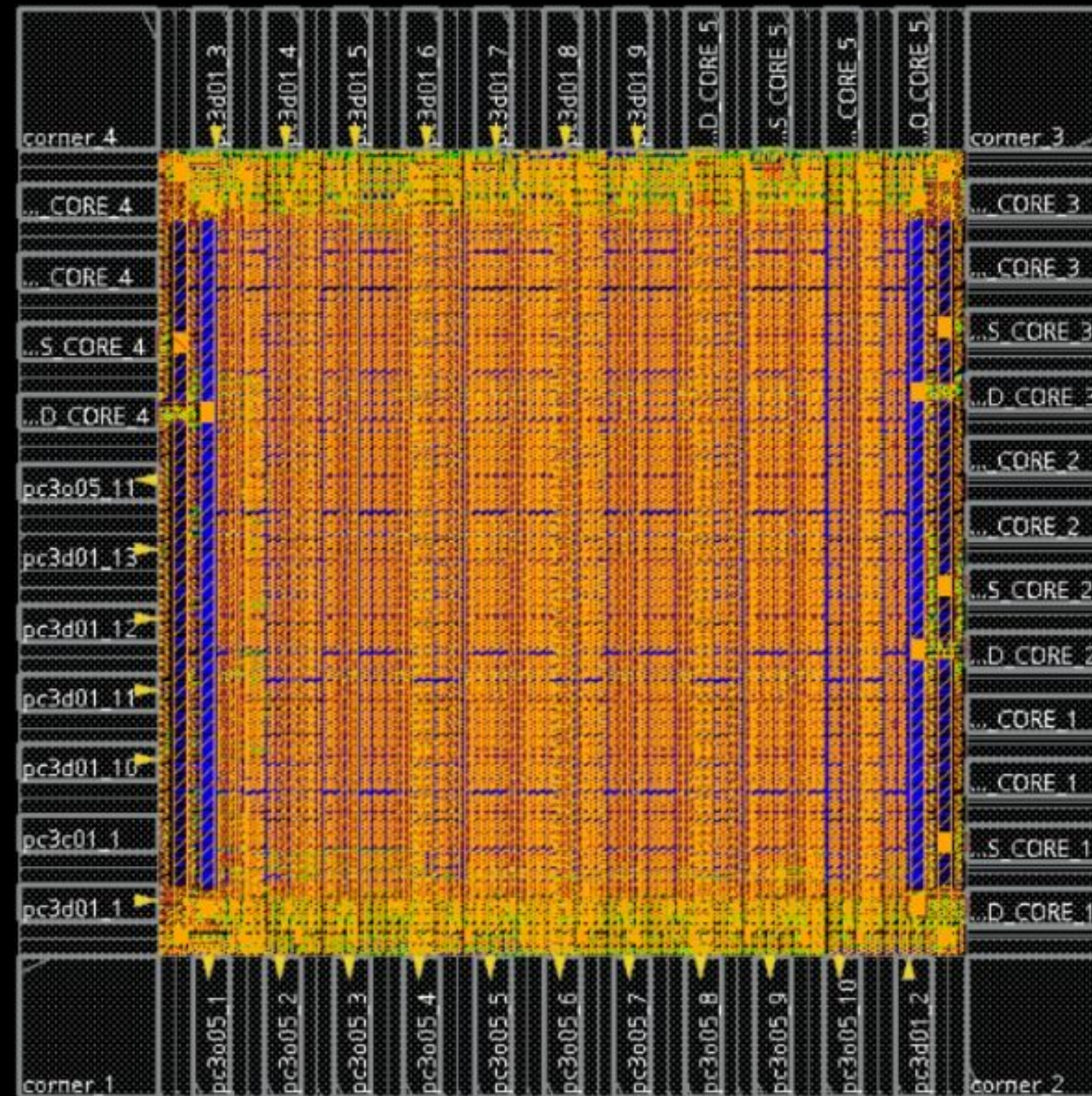
6.PLACEMENT



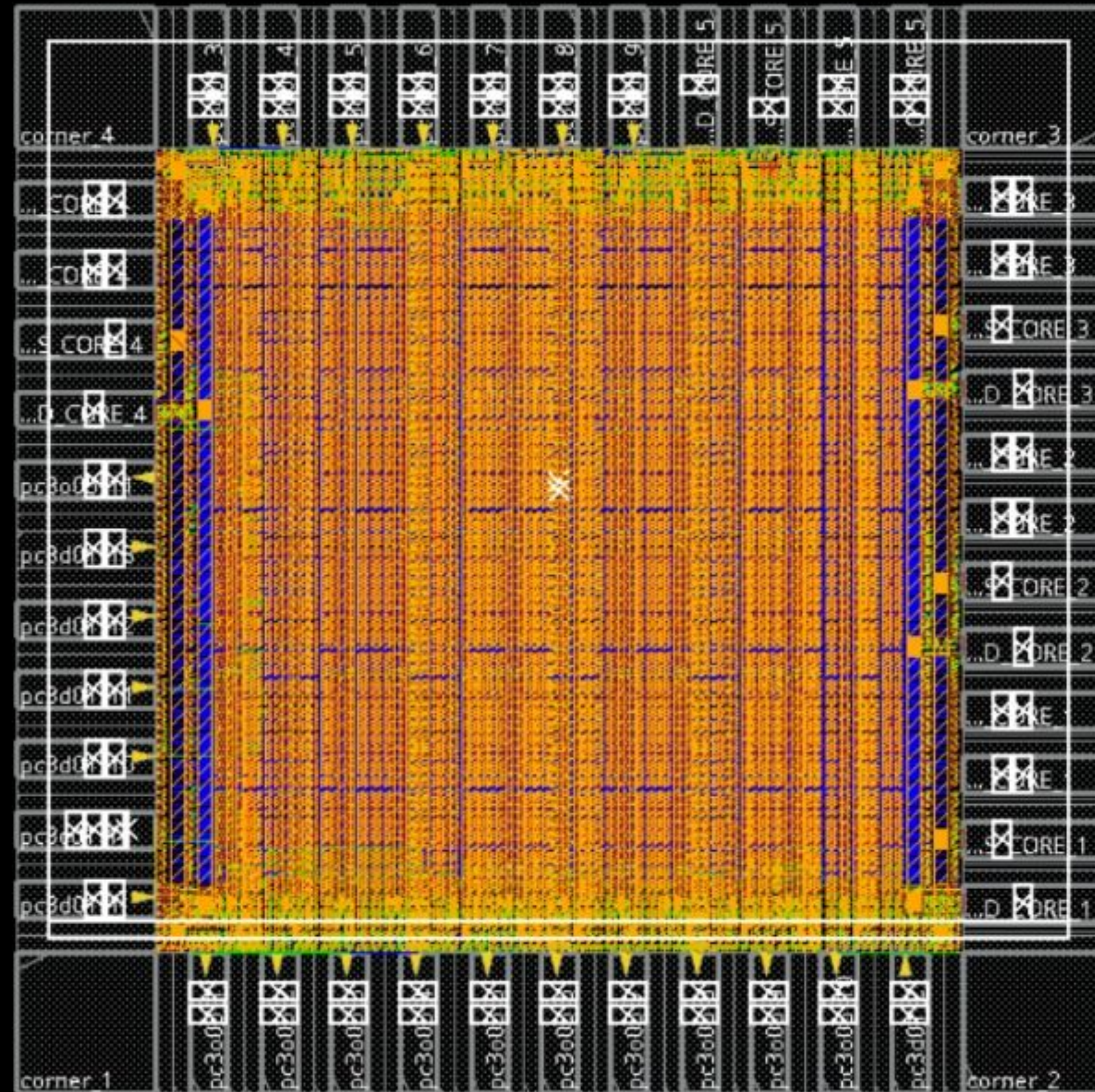
7.CTS

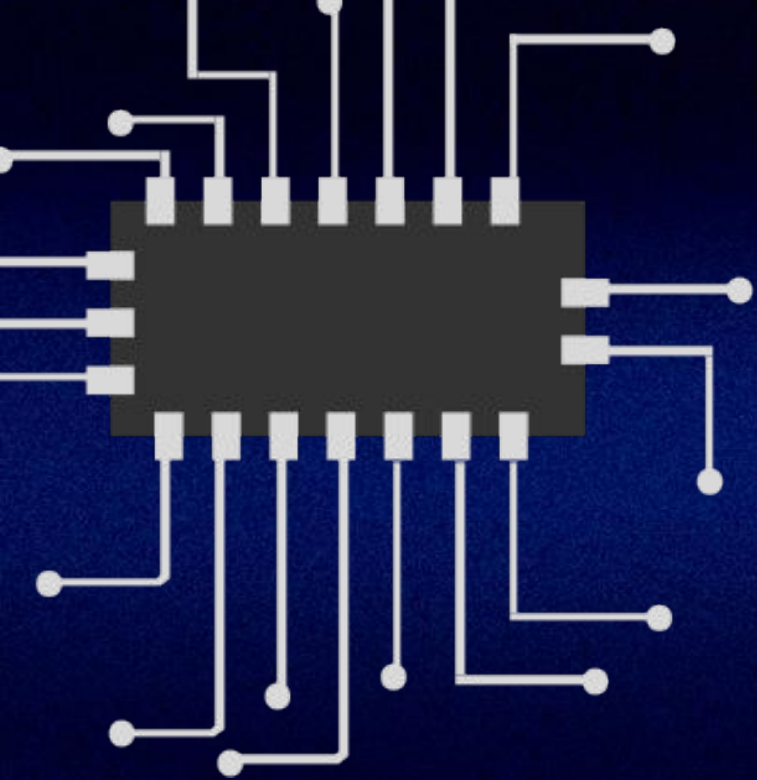


8.ROUTING



9.SIGNOFF





**THANK
YOU**

